

COSC 466/566 Spring 2026

Software (and Web) Security

Doowon Kim (doowon@utk.edu)

Assistant Professor of Computer Science

University of Tennessee, Knoxville

Number Representation

Units of Measurement

- In computers, all data is represented as bits
 - **Bit**: a binary digit, 0 or 1
 - **1 Byte** = a group of 8 bits
- **0b1000100010001000**: 16 bits, or 2 bytes

Hexadecimal

- 4 bits can be represented as 1 hexadecimal digit (base 16)

Binary	Hexadecimal
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7

Binary	Hexadecimal
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F

Hexadecimal

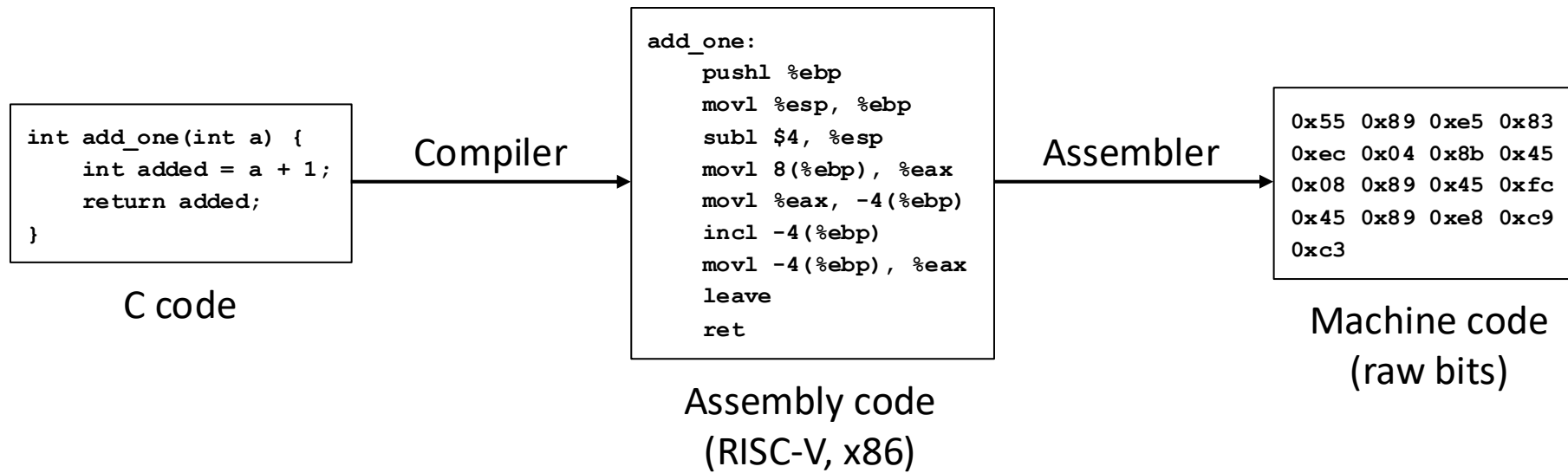
- The byte **0b11000110** can be written as **0xC6** in hex
 - Hard to know what 11000110 is right?
 - But we can split that into 4 bits and 4 bits
 - 1100 = C
 - 0110 = 6
 - Much easier to read C6
- For clarity, we add **0b** in front of bits and **0x** in front of hex

Running C Programs

Compiler, Assembler, Linker, Loader

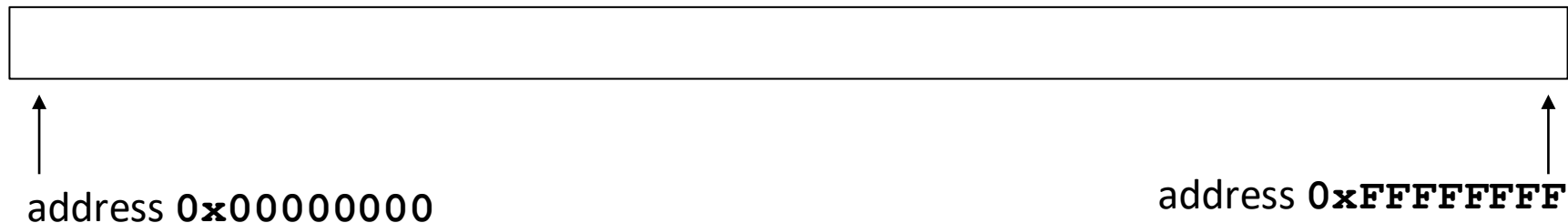
- Compiler: Converts C code into assembly code (RISC-V, x86)
- Assembler: Converts assembly code into machine code (raw bits)
- Linker: Deals with dependencies and libraries
- Loader: Sets up memory space and runs the machine code

Compiler, Assembler, Linker, Loader



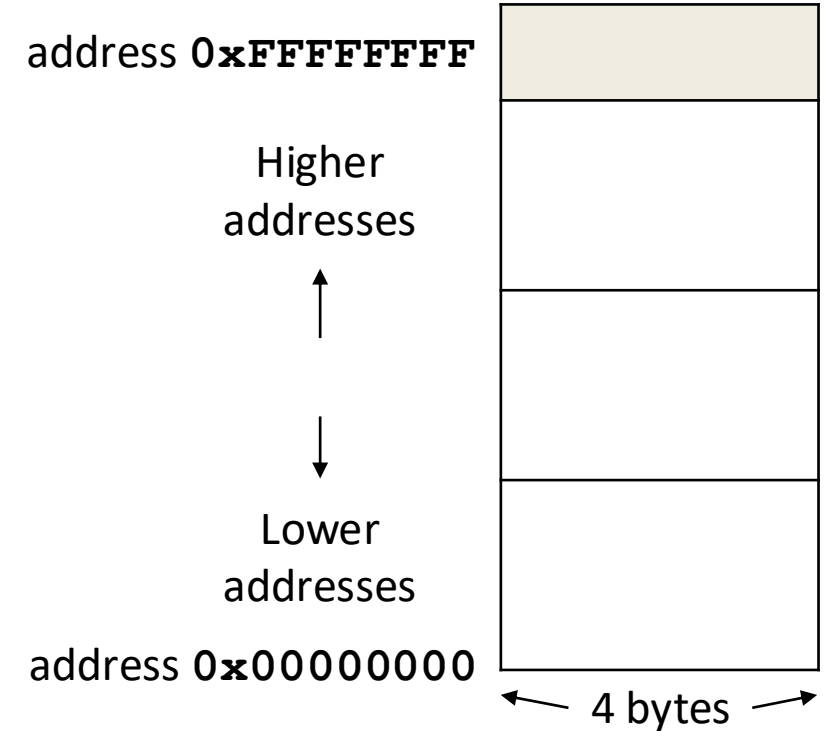
C Memory Layout

- At runtime, the loader tells your OS to give your program a big blob of memory
- On a 32-bit system, the memory has 32-bit addresses
 - On a 64-bit system, memory has 64-bit addresses
 - We use 32-bit systems in this class
- Each address refers to one byte, which means you have 2^{32} bytes of memory



C Memory Layout

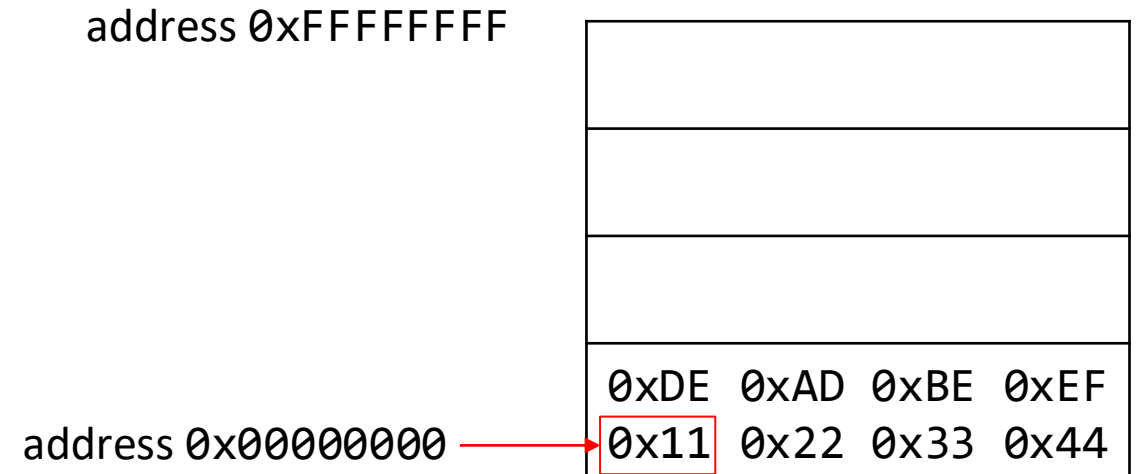
- Often drawn vertically for ease of viewing
 - But memory is still just a long array of bytes



Endianness

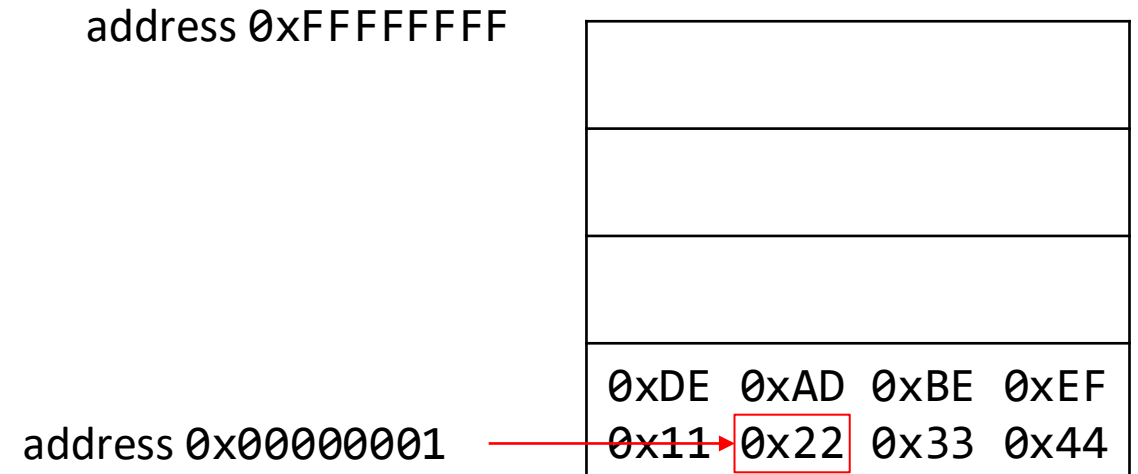
C Memory Layout

- On each row of the grid, we put 4 bytes = 1 word.



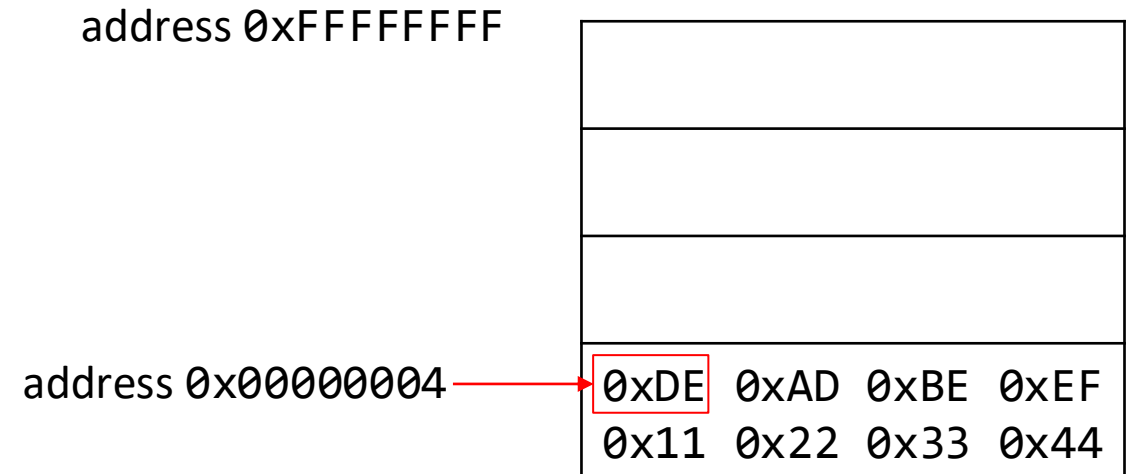
C Memory Layout

- On each row of the grid, we put 4 bytes = 1 word.



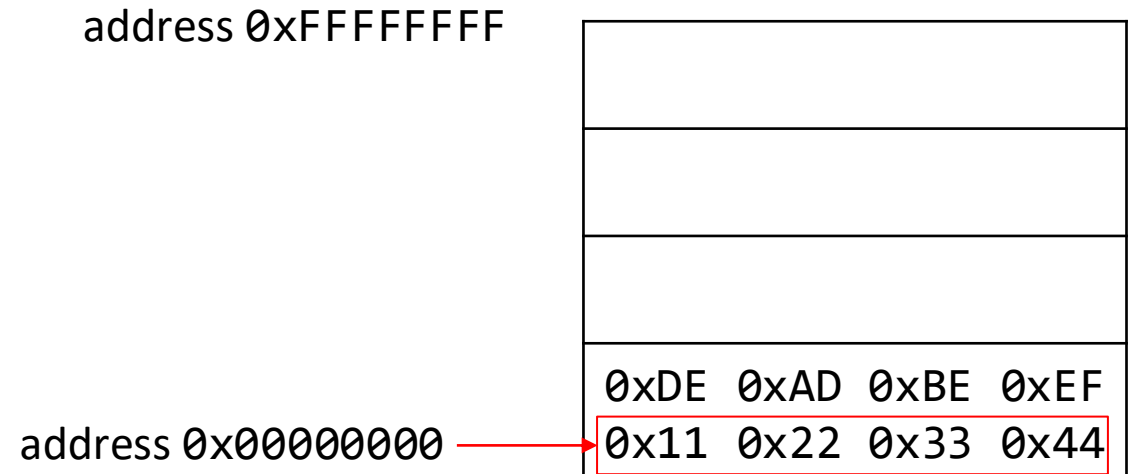
C Memory Layout

- On each row of the grid, we put 4 bytes = 1 word.



Little-endian Words

- On each row of the grid, we put 4 bytes = 1 word.
- We can combine all the bytes on a row to form a word.



Quick Detour: How Do We Write Dates?

- You want to communicate three dates to your friend:
 - January 24, 2025
 - March 13, 2025
 - May 14, 2025
- Challenge: You can only write down nine numbers (no slashes or dashes).

Quick Detour: How Do We Write Dates?

- You want to communicate three dates to your friend:
 - January 24, 2026
 - March 13, 2026
 - May 14, 2026
- Challenge: You can only write down nine numbers (no slashes or dashes).
- One sequence you could try is year-month-day:
 - 26 01 24 26 03 13 26 05 14
- Another sequence you could try is day-month-year:
 - 24 01 26 13 03 26 14 05 26

Quick Detour: How Do We Write Dates?

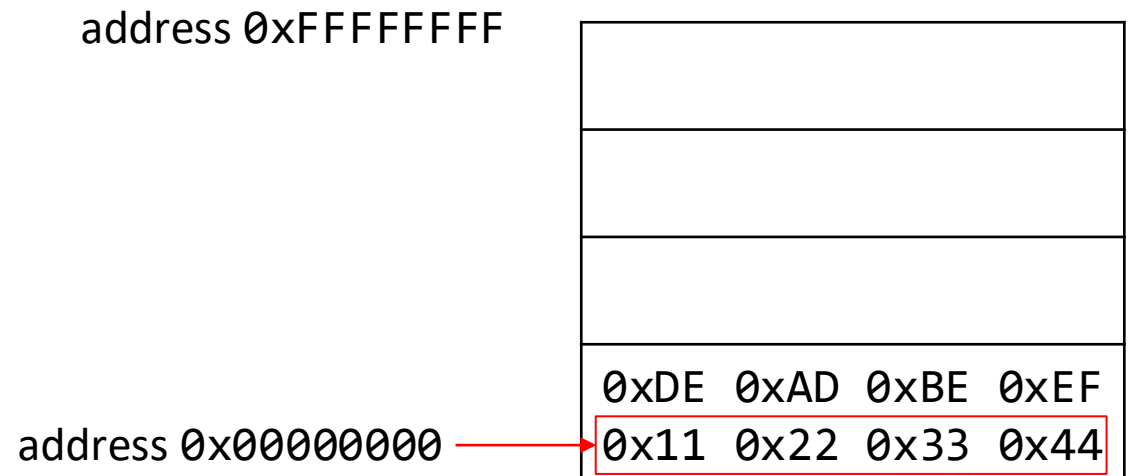
- You want to communicate three dates to your friend:
 - January 24, 2026
 - March 13, 2026
 - May 14, 2026
- Challenge: You can only write down nine numbers (no slashes or dashes).
- One sequence you could try is year-month-day:
 - 26 01 24 26 03 13 26 05 14
- Another sequence you could try is day-month-year:
 - 24 01 26 13 03 26 14 05 26
- Does it matter which one you use?
 - No, as long as your friend knows which one you're using

Quick Detour: How Do We Write Dates?

- 26 01 24 26 03 13 26 05 14 (year-month-day) is the **big-endian** version
 - Because you start with the biggest unit (year)
- 24 01 26 13 03 26 14 05 26 (day-month-year) is the **little-endian** version
 - Because you start with the smallest unit (day)
- In both versions, each group of three numbers is a date. The only difference is how you interpret the three numbers (the order they appear).

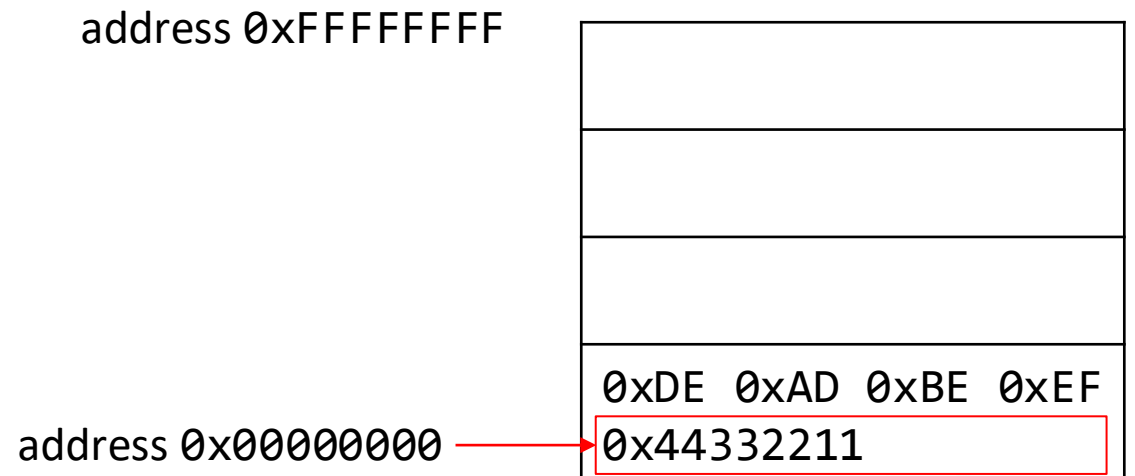
Little-endian Words

- We can combine four bytes on a row to form a word.
- However, x86 is **little-endian**, which means the word formed from the first four bytes is actually `0x44332211`!
- This is just like the dates: each group of 4 bytes is a word. The only difference is how you interpret those bytes (the order they appear).
- Advantage of little-endian?



Notation

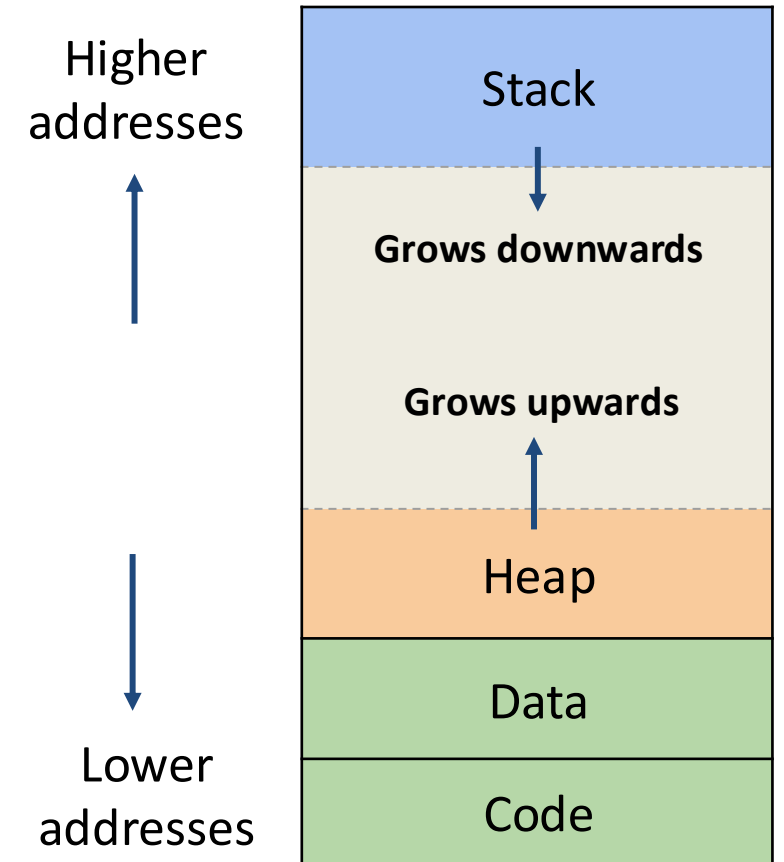
- Usually, we work with words on the stack, so we write each set of 4 bytes as a word.
- But remember that under the hood, the memory is storing the word in “backwards” order.
- This is one of the trickiest parts of the entire unit, so don’t worry if you don’t get it right away.



Memory Layout

x86 Memory Layout

- Code
 - The program code itself (also called “text”)
- Data
 - Static variables, allocated when the program is started
- Heap
 - Dynamically allocated memory using malloc and free
 - As more and more memory is allocated, it grows upwards
- Stack:
 - Local variables and stack frames
 - As you make deeper and deeper function calls, it grows downwards



Registers

- Registers are located on the CPU
- This is different from the memory layout
- Memory: addresses are 32-bit numbers
- Registers are referred to by names (ebp, esp, eip), not addresses

Intro to x86 Architecture

Why x86?

- It's the most commonly used instruction set architecture in consumer computers!
- You are probably using an x86 computer right now...unless you're on a phone, tablet, or recent Mac

x86 Fact Sheet

- Little-endian
 - The least-significant byte of multi-byte numbers is placed at the first/lowest memory address
- Variable-length instructions
 - When assembled into machine code, instructions can be anywhere from 1 to 16 bytes long

x86 Registers

- Each register can store a word and is stored on the CPU (not part of memory)
 - Only 8 main general-purpose registers:
 - EAX, EBX, ECX, EDX, ESI, EDI: General-purpose
 - ESP: Stack pointer
 - EBP: Base pointer
 - We will discuss ESP and EBP in more detail later
- Instruction pointer register: EIP

x86 Syntax

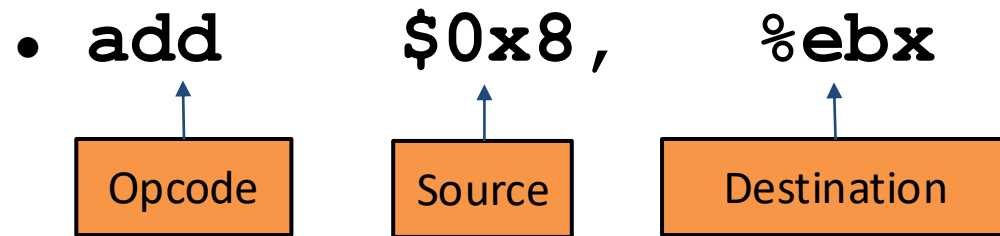
- Register references are preceded with a percent sign %
 - Example: **%eax, %esp, %edi**
- Immediates (constant values) are preceded with a dollar sign \$
 - Example: **\$1, \$161, \$0x4**
 - `movl $0x8048700, %edx`
 - `edx = 0x8048700;`
- Memory references use parentheses and can have immediate offsets
 - Example: `(%esp)` dereferences memory at the address contained in ESP
 - Example: `8(%esp)` dereferences memory 8 bytes above the address contained in ESP

x86 Syntax: Immediate Value vs. Address

- 0x80486e7
 - A number without '\$' prefix is regarded as an **address**
 - In x86 assembly, it refers to the value in that address
- `mov 0x80486e7, %eax`
 - Move the value stored in the address 0x80486fd to `%eax`
 - If 0x80486fd stores 32, then `%eax = 32`;

x86 Assembly

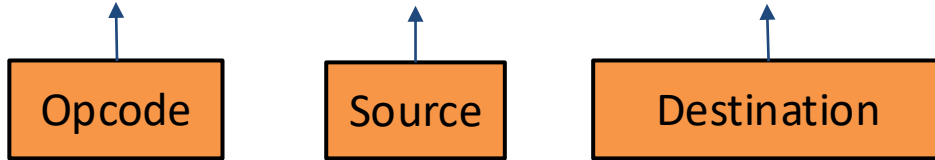
- Instructions are composed of an opcode and zero or more operands.



- Pseudocode: **EBX = EBX + 0x8**
- The destination comes last
- The **add** instruction only has two operands; and the destination is an input
- This instruction uses a register and an immediate

x86 Assembly

- `xorl 4(%esi), %eax`



- Pseudocode: $\text{EAX} = \text{EAX} \wedge * (\text{ESI} + 4)$
- This is a memory reference, where the value at 4 bytes above the address in ESI is dereferenced, XOR'd with EAX, and stored back into EAX

x86 Assembly (AT&T/GNU Syntax)

- [instruction] [source] [destination]
- `mov %esp, %ebp` `mov %esp, %ebp`
 - Assign the value of esp to ebp, i.e., `ebp = esp;` in C
- `mov $0x0, %eax` `movl $0x0, -0x4(%ebp)`
 - Assign value 0 to eax, i.e., `eax = 0;` in C
 - Assign value 0 to the address pointed by ebp-4, i.e., `ebp[-1] = 0;` (ebp as int *)
- [instruction] [destination]
- `call 0x80483c0 <printf@plt>` `call 0x80483c0 <printf@plt>`
 - Call `printf()`
- `jne 0x804861f <main+127>` `jne 0x804861f <main+127>`
 - Jump to 0x804861f if the comparison result was Not Equal (NE)

OP A B

B = A OP B

add %eax, %ebx

ebx = eax + ebx;

x86 Assembly (AT&T/GNU Syntax)

- mov: move value from src to dst
 - mov %esp, %ebp (ebp = esp;)
 - movl: move 4 bytes (move long integer)
 - movw: move 2 bytes (move word integer)
 - movb: move 1 byte (move byte integer)
- call: call the destination function
 - call 0x80483c0 <printf@plt>
 - call 0x80483b0 <strcmp@plt>
- ret: return from a function call
 - ret 0x0804863d <+157>: ret

x86 Assembly (AT&T/GNU Syntax)

- Arithmetic Operations
 - `add %eax, %ebx` (`ebx = eax + ebx;`)
 - `add $1, %eax` (`eax = eax + 1;`)
 - `sub $0x228, %esp` (`esp = esp - 0x228;`)
 - `xor %eax, %eax` (`eax = eax ^ eax;`)

x86 Assembly (AT&T/GNU Syntax)

- Compare
 - `cmp $0x0,%eax` **`cmp $0x0,%eax`**
 - Compare the value 0x0 to the value of %eax
 - Comparison result will be stored in the EFLAGS register
 - Will be used in the jump instruction
- Jump
 - `je: jump if equal`
 - If (eax == 0)
 - `jne: jump if not equal` **`jne 0x804861f <main+127>`**
 - if (eax != 0)
 - `jg: jump if greater`
 - If (eax > 0)
 - `jge: jump if greater and equal`
 - If (eax >= 0)
 - `Jmp: jump anyway!` **`jmp 0x8048633 <main+147>`**

Dereferencing in x86 Assembly

- Parenthesis means accessing the register as an address
- `mov %eax, (%esp)` **`mov %eax, (%esp)`**
 - Regard `%esp` as `int *esp`; in C
 - `*esp = eax;` (move the value of `eax` to the address pointed by `%esp`)
 - `esp[0] = eax;`
- `movl $0x0, -0x4(%ebp)` **`movl $0x0, -0x4(%ebp)`**
 - Move 0 to `(%ebp-4)`
 - Regard `%ebp` as `int *ebp`; in C, `-1` to `int*` is indeed `-4` in address
 - `*(ebp-1) = 0;`
 - `ebp[-1] = 0;`

Example

```
void swap (long *xp, long *yp) {  
    long t0 = *xp;  
    long t1 = *yp;  
    *xp = t1;  
    *yp = t0;  
}
```

Register	Value
%rdi	xp
%rsi	yp
%rax	t0
%rdx	t1

swap:

```
movq    (%rdi), %rax  
movq    (%rsi), %rdx  
movq    %rdx, (%rdi)  
movq    %rax, (%rsi)  
ret
```

Address	Value
xp	100
xy	200

Example

```
void swap (long *xp, long *yp) {  
    long t0 = *xp;  
    long t1 = *yp;  
    *xp = t1;  
    *yp = t0;  
}
```

Register	Value
%rdi	0x1234
%rsi	0x1238
%rax	????
%rdx	????

swap:

```
movq    (%rdi), %rax  
movq    (%rsi), %rdx  
movq    %rdx, (%rdi)  
movq    %rax, (%rsi)  
ret
```

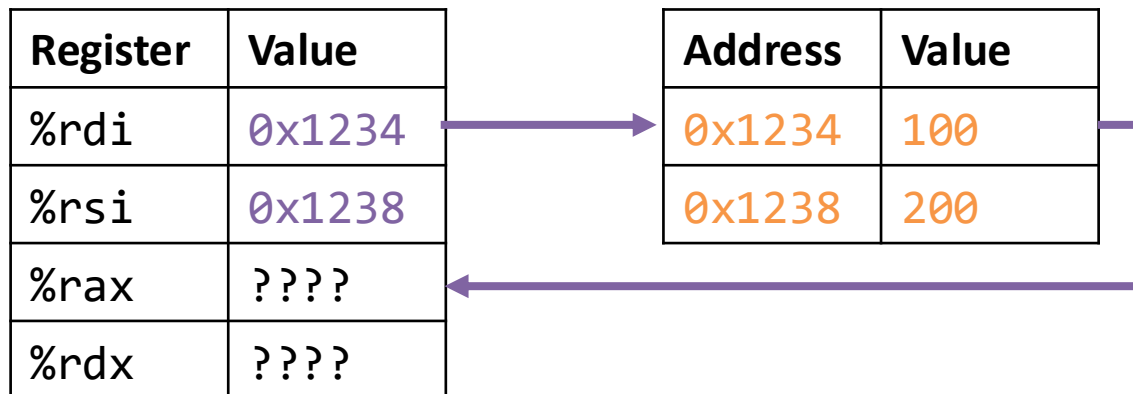
Address	Value
0x1234	100
0x1238	200

Example

```
void swap (long *xp, long *yp) {  
    long t0 = *xp;  
    long t1 = *yp;  
    *xp = t1;  
    *yp = t0;  
}
```

swap:

```
movq    (%rdi), %rax  
movq    (%rsi), %rdx  
movq    %rdx, (%rdi)  
movq    %rax, (%rsi)  
ret
```

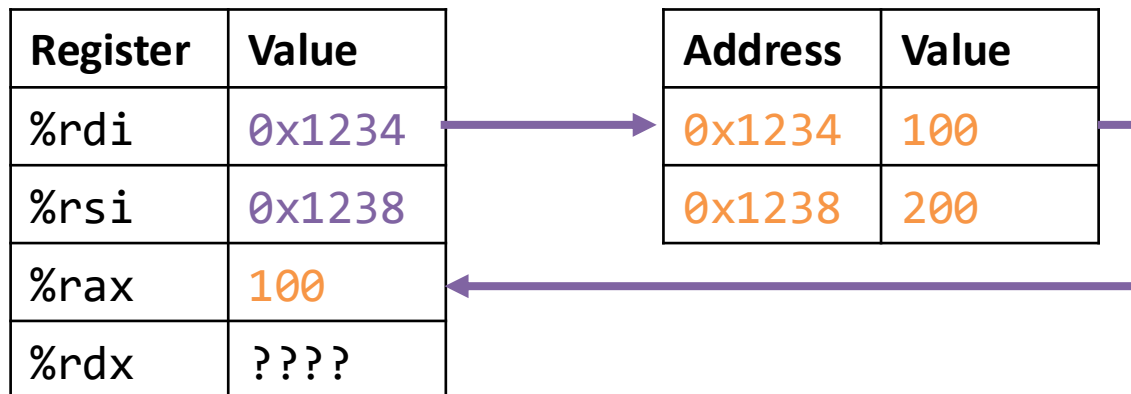


Example

```
void swap (long *xp, long *yp) {  
    long t0 = *xp;  
    long t1 = *yp;  
    *xp = t1;  
    *yp = t0;  
}
```

swap:

```
movq    (%rdi), %rax  
movq    (%rsi), %rdx  
movq    %rdx, (%rdi)  
movq    %rax, (%rsi)  
ret
```

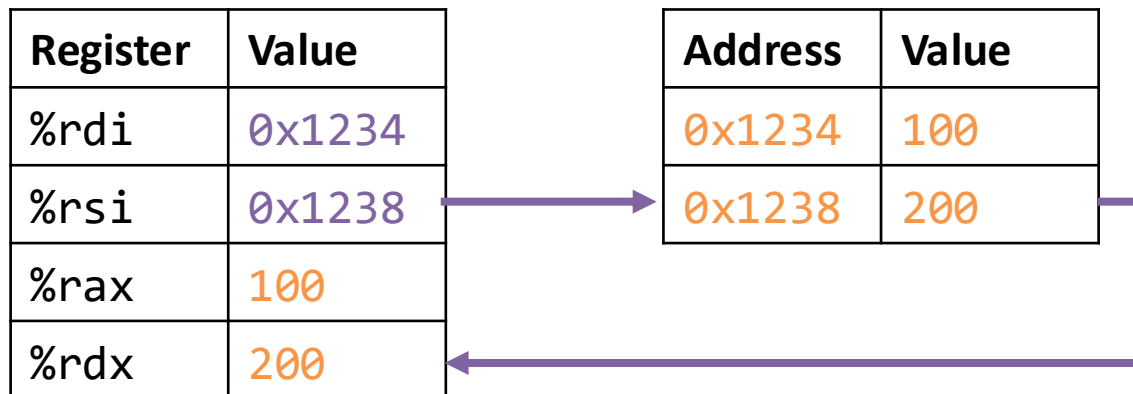


Example

```
void swap (long *xp, long *yp) {  
    long t0 = *xp;  
    long t1 = *yp;  
    *xp = t1;  
    *yp = t0;  
}
```

swap:

```
movq    (%rdi), %rax  
movq    (%rsi), %rdx  
movq    %rdx, (%rdi)  
movq    %rax, (%rsi)  
ret
```

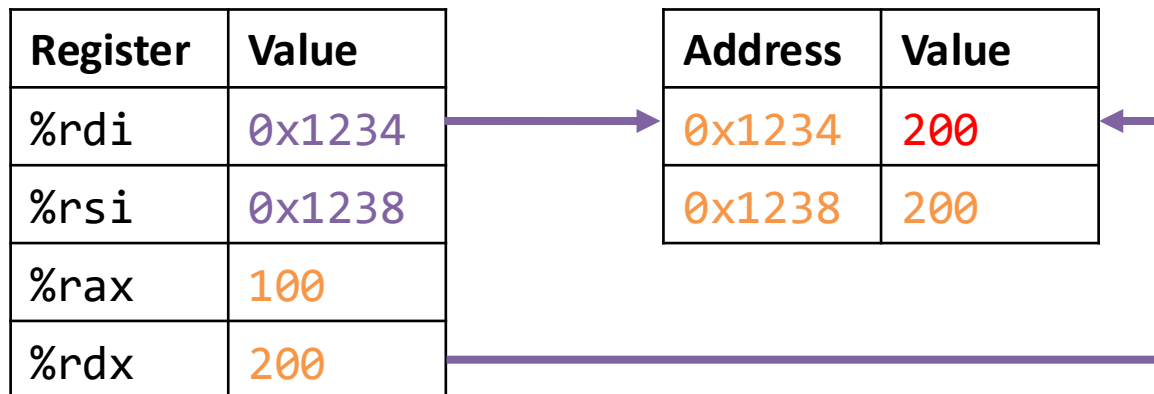


Example

```
void swap (long *xp, long *yp) {  
    long t0 = *xp;  
    long t1 = *yp;  
    *xp = t1;  
    *yp = t0;  
}
```

swap:

```
movq    (%rdi), %rax  
movq    (%rsi), %rdx  
movq    %rdx, (%rdi)  
movq    %rax, (%rsi)  
ret
```



Example

```
void swap (long *xp, long *yp) {  
    long t0 = *xp;  
    long t1 = *yp;  
    *xp = t1;  
    *yp = t0;  
}
```

swap:

```
movq    (%rdi), %rax  
movq    (%rsi), %rdx  
movq    %rdx, (%rdi)  
movq    %rax, (%rsi)  
ret
```

